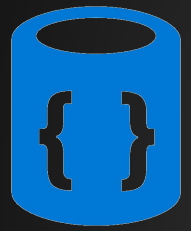
Architecture NoSQL

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Objectif du cours



Comprendre les enjeux derrière les NoSQL, les cas d'utilisations et les différences avec le SQL

En groupe, vous répondrez à une problématique d'entreprise afin de concevoir la meilleure architecture de DB en fonction des besoins & des capacité

A l'oral (de 10m), vous justifierez vos choix, présenterez un budget et des deadlines

1. Le NoSQL

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Le NoSQL

- ***Raisons de l'apparition du NoSQL***
  - Le **NoSQL** (**N**ot **O**nly **SQL**) répond à un besoin de performance, de disponibilité et de flexibilité dû aux big datas & aux applications web modernes
  - Il apporte de nouveaux schémas, de nouvelles possibilités d'optimisation et d'adaptation face à des situations nouvelles dû à notre utilisation d'internet
  - Il va en résulter un changement de paradigme dans la création d'une DB et de ses logiques d'utilisation. La rigidité est déplacée au niveau de l'application et de l'écriture, et non pas au moment de l'insertion dans la DB

Le NoSQL

- ***Changements dans la logique / paradigme***
  - Dans cette nouvelle logique, on va manipuler de la donnée dans le cadre de DB semi-structurées et/ou non-structurées
  - Ainsi, on passe d'une logique **ACID**, mettant l'emphase sur la cohérence, à une logique **BASE**, avançant une logique de disponibilité de la donnée.
  - Grâce à ces changements, la scalabilité horizontale est bien plus accessible et peut donc être envisagée plus fréquemment.

Le NoSQL

- ***Les scalabilités***
  - La **scalabilité verticale** consiste à augmenter les propriétés physiques d'un serveur. Ainsi, on augmentera la taille du disque, la RAM, le CPU etc.. Cette approche ne répond pas aux problématiques de disponibilité et ne peut croître indéfiniment.
  - La **scalabilité horizontale** consiste à multiplier les noeuds (instances serveurs). Les noeuds font partie d'un cluster d'une DB. Chaque noeuds aura un ou des shards primaires et/ou réplicas. Les données de la DB resteront accessibles grâce à une clé de sharding. Un shard est une partie de la DB duquel le noeud attaché est responsable pour les requêtes d'écriture & de lecture.

Le NoSQL

- ***Les clés de sharding***
  - Différentes clés peuvent exister
    - Simple (uniquement par l'ID)
    - Géographique (pays ou région ou combiné)
    - Par hash, c'est-à-dire qu'on réparti en fonction du nombre de donnée présente sur le shard
  - Il est possible de les combiner afin de gérer selon les localités, besoins métiers ou autre
  - Une mauvaise clé de sharding peut créer des hotspots en redirigeant trop de donnée faire un shard

2. Théorème CAP & Approches ACID/BASE

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

CAP - ACID & BASE

- ***Principes du Théorème***
  - Énonce les contraintes qu'il y a dans les systèmes distribués (environnement où plusieurs appareils effectuent des actions sur un réseau)
  - **C** (Consistency) : Avoir la même donnée au même moment pour les utilisateurs
  - **A** (Availability) : Les requêtes doivent pouvoir s'effectuer à chaque moment
  - **P** (Partition Tolerance) : S'il y a des pertes dans une ou des parties du réseau, le système continue de fonctionner
  - Les contraintes révèlent une nécessité de faire des arbitrages afin de favoriser une contrainte au détriment d'une autre

CAP - ACID & BASE

- ***ACID***
  - Approche utilisé dans les SGBDR afin d'assurer de la cohérence et de la stabilité dans le système
  - **Atomicity** : Une requête est entièrement exécutable ou alors elle rollback
  - **Cohérence** : Après la requête, la DB passe d'un état valide à un autre état valide. Cela assure la cohérence des données pour l'ensemble des users
  - **Isolation** : Simule une séquentialité des requêtes afin que différents states de la DB ne s'écrasent pas. Assure la cohérence
  - **Durability** : Assure la persistance des données même après un problème technique, du moment où le commit de la requête est passé. Utilisation de disques et de journaux de transactions (reconstitution de requête si besoin)

CAP - ACID & BASE

- ***BASE***
  - Approche associée aux solutions NoSQL. Plus flexible et permissive que l'approche ACID. Elle est néanmoins moins fiable mais permet une meilleure performance et gestion des contraintes.
  - **BA** (Basically Available) : Privilégie la disponibilité plutôt que la cohérence. Cette philosophie octroie des temps de réponse plus rapides.
  - **S** (Soft State) : L'état du système peut être en train d'évoluer durant la requête sans même une intervention explicite
  - **E** (Eventually Consistent) : Assure qu'après une synchronisation / état de transition, l'état reviendra vers la cohérence des données.

3. Les approches NoSQL

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Les approches NoSQL

- ***Les approches***
  - **Clé-valeur** : Identifiant unique d'une clé permettant d'accéder à une valeur non contrôlée par la DB et qui n'a pas besoin d'être structurée. La rapidité est augmentée par le fait de ne regarder que la clé associée mais cela empêche des requêtes plus lourdes et complexes
  - **Orientés Documents** : Stockage sous forme de "Documents" comme des JSON / BSON / XML ou autre Binary Documents. Permet des schémas flexibles ce qui aide pour des données changeantes. Il y a tout une logique d'application propre à la création des documents, notamment dans le cadre d'embedding et de referencing

Les approches NoSQL

- ***Les approches***
  - **Orientés Colonnes** : Gestion de la DB et des données par column et non par row. Les datas sont gérés dans des “column families”. Lors d’une recherche, chaque ligne pourra avoir un nombre variable de colonnes. Utile dans la gestion des volumes massifs de datas comme les logs.
  - **Orientés Graphes** : Modèle de DB qui fonctionne sous forme de “noeuds” d’informations. Permet de récupérer efficacement des données voisines. Très utile dans le cadre des réseaux sociaux ou des systèmes de recommandations. On récupère les amis d’amis ou des produits pouvant nous intéressés en fonction de ce qu’on a acheté

4. Causes et conséquences des choix d’architecture

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Causes & Conséquences architecture

- ***Variables impactant l'arbitrage***
  - **Nature et volume des données**. A quel point les données sont importantes et à quel point elles sont structurées ?
  - **Performance**. La DB a-t-elle un grand besoin de réactivité et de délai de réponse faible ?
  - **Cohérence**. A quelle point la cohérence des données est importante ?
  - **Capacités humaines**. L'équipe a-t-elle les compétences ? Si non, les acquérir prendrait-il beaucoup de temps ?
  - **Capacités financières**. Coût des licences / infrastructure / compétences internes

Causes & Conséquences architecture

- ***Potentiels problèmes***
  - **Utiliser une approche sans la comprendre**. Reproduire une DB SQL classique dans une approche NoSQL. On a tous les inconvénients sans les bénéfices
  - **La Structure flexible**. C'est un avantage à fort potentiel d'erreur. Flexible ne veut pas dire sans structure ou inconsistent. L'obligation passe de la DB à une rigueur de la part des développeurs
  - **Mauvaise utilisation des clés de sharding** dans le cadre d'une scalabilité horizontale. Dans ce cadre, cela peut créer des hotspots à cause d'une mauvaises répartitions des données et créer de gros ralentissements.

5. Le projet

Nicolas Triau
Mail : nicolas.triau@yahoo.fr
Discord : gillesleveau

Le projet

- ***Consignes***
  - Groupe de 1 à 3-(4) personnes
  - Traduire les besoins de l'entreprise
  - Justifier les choix de DB
  - Anticiper les points de frictions possibles pour l'établissement de des solutions proposées
  - Établir une roadmap
  - Établir un budget
  - Établir les besoins RH
  - Présentation oral à destination d'un public non-tech. Adapter le langage tout en signifiant bien les enjeux face à l'implémentation d'une architecture NoSQL dans le projet
  - Choisir l'un des 3 cas

Le projet

- ***Consignes***
  - L'oral se fera le 31/03 à partir de 14/14h30
  - L'ordre des groupes sera au hasard
  - Oral de 10 minutes | 5 à 10 minutes de questions
  - L'oral s'arrête si vous dépassez les 10 minutes
  - Le timer commence même si vous n'êtes pas prêt
  - Barème :
    - 8 pts sur la solution apportée
    - 6 pts sur la présentation du PowerPoint
    - 6 pts sur l'oral
  - Vous inscrivez votre groupe ainsi que votre sujet sur le XLSX et vous déposer vos présentation avant ***30/03 - 23h59***

Le projet

https://drive.google.com/drive/folders/1B5BMK_33zX6VnY-3fuE9fM0OU00-ExFK?usp=sharing

Le projet

- ***1er cas***
  - Startup e-commerce
  - Base actuelle 100% SQL. Que ce soit produits, recommendations, clients, fournisseurs etc…
  - Ralentissements lors de fort trafics
  - La direction prévoit une deadline pour dans 6 mois en raison du Black Friday
  - Le plus gros problème est le ralentissement sur le front
  - Ouverture à d’autres pays d’ici 1 an

Le projet

- ***2eme cas***
  - App de livraisons de repas
  - Base actuelle 100% SQL
  - Pas de temps réel pour la livraison car l'actualisation se fait trop lentement. Le temps réel est une vraie contrainte. Lorsqu'un restaurant ferme, l'information doit se diffuser la plus rapidement possible. Idem pour la localisation du livreur
  - Ralentissements lors de fort trafics
  - Chaque restaurant précise ou non des choses sur ses produits, la fiche produit est donc inconsistante
  - La direction prévoit une deadline d'1 an

Le projet

- ***3eme cas***
  - Plateforme JV (type Steam)
  - Base actuelle 100% SQL
  - Toutes les actualisations sont très lentes
  - Gestion des produits ainsi que des pages Workshops associées
  - Système de recommandations très important, notamment via la friendlist
  - Les Informations de sessions sont importantes pour les succès & temps de jeu
  - Gestions de logs est un enjeu en raison des différents crashs des PC selon le jeu & la date, afin de pouvoir les donner aux studios
  - Deadline 8 mois pour tout changer, sans pression extérieure